4.2 Recurrent neural networks

- in n-gram models, the conditional probability of word x_t at time step t only depends on the $n-1$ previous words
- if we want to incorporate the possible effect of words earlier than time step $t-(n-1)$ on x_t, we need to increase n
- however, the number of model parameters would also increase exponentially with it, as we need to store $|\mathcal{V}|^n$ numbers for a vocabulary set $\mathcal{V}$
- hence, rather than modeling $P(x_t|x_{t-1},\ldots,x_{t-n+1})$, it is preferable to use a latent variable model:

$$P(x_t|x_{t-1},\ldots,x_1) \approx P(x_t|h_{t-1}),$$

  where h_{t-1} is a *hidden state* (also known as a *hidden variable*) that stores the sequence information up to time step $t-1$
- in general, the hidden state at any time step t could be computed based on both the current input x_t, and the previous hidden state h_{t-1}:

$$h_t = f(x_t, h_{t-1}). \tag{1}$$

4.2 Recurrent neural networks

- for a sufficiently powerful function f in (1), the latent variable model is not an approximation
- after all, h_t may simply store all the data it has observed so far
- however, it could potentially make both computation and storage expensive
- recall that we have discussed hidden layers with hidden units in Chapter 2
- it is noteworthy that hidden layers and hidden states refer to two very different concepts
- hidden layers are, as explained, layers that are hidden from view, on the path from input to output
- hidden states are, technically speaking, *inputs* to whatever we do at a given step, and they can only be computed by looking at data at previous time steps

4.2 Recurrent neural networks

- *recurrent neural networks* (*RNNs*) are neural networks with hidden states
- before introducing the RNN model, we first revisit the MLP model introduced in Chapter 2
- let us take a look at an MLP with a single hidden layer
- let the hidden layer's activation function be ϕ
- given a mini-batch of examples $\boldsymbol{X} \in \mathbb{R}^{n \times d}$ with batch size n and d inputs, the hidden layer's output $\boldsymbol{H} \in \mathbb{R}^{n \times h}$ is calculated as:

$$\boldsymbol{H} = \phi(\boldsymbol{X}\boldsymbol{W}_{xh} + \boldsymbol{b}_h). \tag{2}$$

- in (2), we have the weight parameter $\boldsymbol{W}_{xh} \in \mathbb{R}^{d \times h}$, the bias parameter $\boldsymbol{b}_h \in \mathbb{R}^{1 \times h}$, and the number of hidden units h, for the hidden layer
- thus, broadcasting is applied during the summation

4.2 Recurrent neural networks

- next, the hidden variable $\boldsymbol{H}$ is used as the input of the output layer
- the output layer is given by:

$$\boldsymbol{O} = \boldsymbol{H}\boldsymbol{W}_{hq} + \boldsymbol{b}_q,$$

  where $\boldsymbol{O} \in \mathbb{R}^{n \times q}$ is the output variable, $\boldsymbol{W}_{hq} \in \mathbb{R}^{h \times q}$ is the weight parameter, and $\boldsymbol{b}_q \in \mathbb{R}^{1 \times q}$ is the bias parameter of the output layer
- if it is a classification problem, we can use softmax($\boldsymbol{O}$) to compute the probability distribution of the output categories
- this is entirely analogous to the regression problem we discussed previously in Section 4.1
- things are entirely different when we have hidden states
- let us look at the structure in some more detail

4.2 Recurrent neural networks

- assume that we have a mini-batch of inputs $\boldsymbol{X}_t \in \mathbb{R}^{n \times d}$ at time step t
- in other words, for a mini-batch of n sequence examples, each row of $\boldsymbol{X}_t$ corresponds to one example at time step t from the sequence
- next, denote by $\boldsymbol{H}_t \in \mathbb{R}^{n \times h}$ the hidden variable of time step t
- unlike the MLP, here we save the hidden variable $\boldsymbol{H}_{t-1}$ from the previous time step and introduce a new weight parameter $\boldsymbol{W}_{hh} \in \mathbb{R}^{h \times h}$ to describe how to use the hidden variable of the previous time step in the current time step
- specifically, the calculation of the hidden variable of the current time step is determined by the input of the current time step, together with the hidden variable of the previous time step:

$$\boldsymbol{H}_t = \phi(\boldsymbol{X}_t \boldsymbol{W}_{xh} + \boldsymbol{H}_{t-1} \boldsymbol{W}_{hh} + \boldsymbol{b}_h). \tag{3}$$

4.2 Recurrent neural networks

- compared with (2), (3) adds one more term $\boldsymbol{H}_{t-1}\boldsymbol{W}_{hh}$, and thus instantiates (1)
- from the relationship between hidden variables $\boldsymbol{H}_t$ and $\boldsymbol{H}_{t-1}$ of adjacent time steps, we know that these variables captured and retained the sequence's historical information up to their current time step, just like the state or memory of the neural network's current time step
- therefore, such a hidden variable is called a *hidden state*
- since the hidden state uses the same definition of the previous time step in the current time step, the computation of (3) is *recurrent*
- hence, neural networks with hidden states based on recurrent computation are named *recurrent neural networks* (*RNNs*)
- layers that perform the computation of (3) in *RNNs* are called *recurrent layers*

4.2 Recurrent neural networks

- there are many different ways for constructing RNNs; RNNs with a hidden state defined by (3) are very common
- for time step t, the output of the output layer is similar to the computation in the MLP:

$$\boldsymbol{O}_t = \boldsymbol{H}_t \boldsymbol{W}_{hq} + \boldsymbol{b}_q.$$

- parameters of the RNN include the weights $\boldsymbol{W}_{xh} \in \mathbb{R}^{d \times h}$, $\boldsymbol{W}_{hh} \in \mathbb{R}^{h \times h}$, and the bias $\boldsymbol{b}_h \in \mathbb{R}^{1 \times h}$ of the hidden layer, together with the weights $\boldsymbol{W}_{hq} \in \mathbb{R}^{h \times q}$, and the bias $\boldsymbol{b}_q \in \mathbb{R}^{1 \times q}$ of the output layer
- it is worth mentioning that, even at different time steps, RNNs always use these same model parameters
- therefore, the parameterization cost of an RNN does not grow as the number of time steps increases

4.2 Recurrent neural networks

- Figure 3 illustrates the computational logic of an RNN at three adjacent time steps
- at any time step t, the computation of the hidden state can be treated as:
  - concatenating the input $\boldsymbol{X}_t$ at the current time step t and the hidden state $\boldsymbol{H}_{t-1}$ at the previous time step $t-1$
  - feeding the concatenation result into a fully-connected layer with the activation function ϕ
- the output of such a fully-connected layer is the hidden state $\boldsymbol{H}_t$ of the current time step t

4.2 Recurrent neural networks

- in this case, the model parameters are the concatenation of $\boldsymbol{W}_{xh}$ and $\boldsymbol{W}_{hh}$, and a bias of $\boldsymbol{b}_h$, all from (3)
- the hidden state of the current time step t, $\boldsymbol{H}_t$, will participate in computing the hidden state $\boldsymbol{H}_{t+1}$ of the next time step $t+1$
- what is more, $\boldsymbol{H}_t$ will also be fed into the fully-connected output layer to compute the output $\boldsymbol{O}_t$ of the current time step t
- so, the calculation of $\boldsymbol{X}_t\boldsymbol{W}_{xh} + \boldsymbol{H}_{t-1}\boldsymbol{W}_{hh}$ for the hidden state is equivalent to matrix multiplication between the concatenation of $\boldsymbol{X}_t$ and $\boldsymbol{H}_{t-1}$ and the concatenation of $\boldsymbol{W}_{xh}$ and $\boldsymbol{W}_{hh}$

4.2 Recurrent neural networks

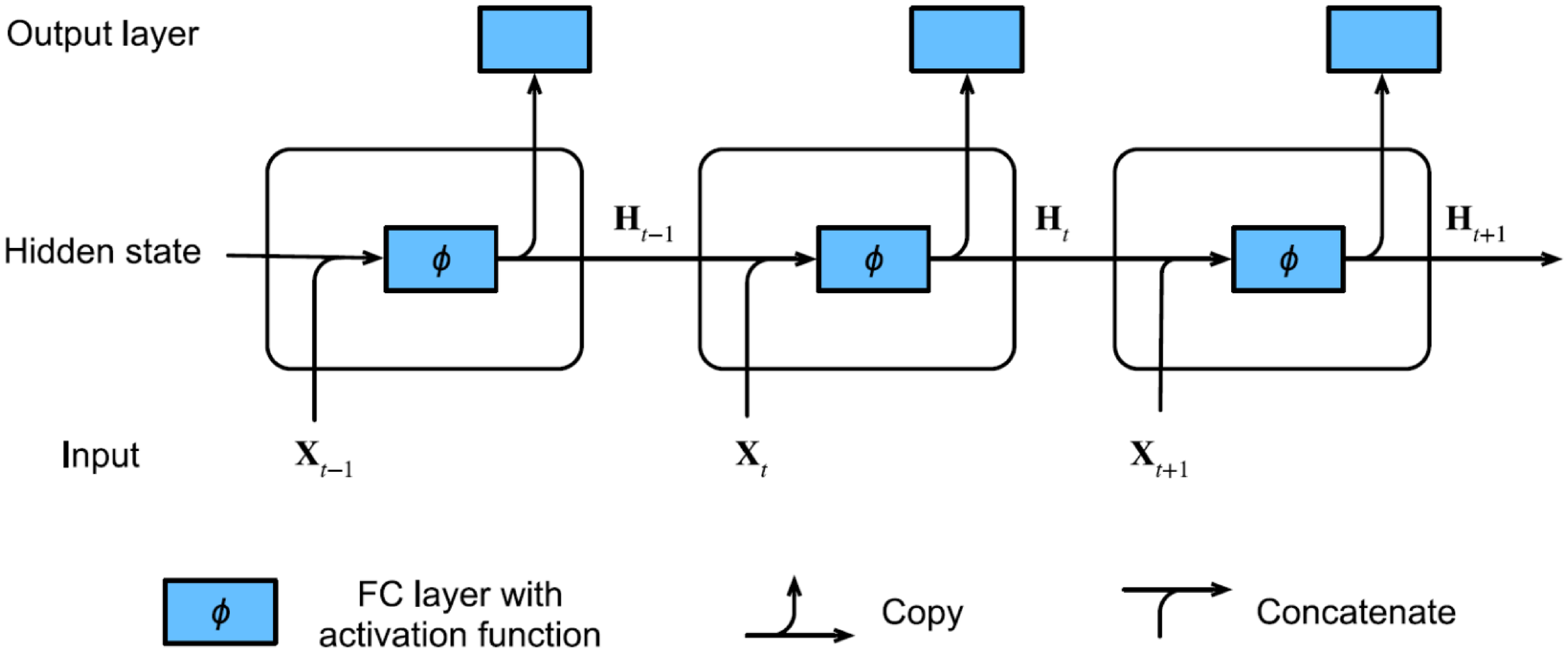


Figure 3: An RNN with a hidden state.

4.2 Recurrent neural networks

- for a sequence of length T, we will compute the gradients over T time steps in an iteration, which results in a chain of matrix products with length $\mathcal{O}(T)$, during backpropagation
- as mentioned in Chapter 2, this might result in numerical instability, e.g., the gradients may either explode or vanish, when T is large
- therefore, RNN models often need extra help to stabilize the training

4.2 Recurrent neural networks

- generally speaking, when solving an optimization problem, we take update steps for the model parameter, say in the vector form $\boldsymbol{x}$, in the direction of the negative gradient $\boldsymbol{g}$, on a mini-batch
- for example, with $\eta > 0$ as the learning rate, in one iteration, we update $\boldsymbol{x}$ as $\boldsymbol{x} - \eta \boldsymbol{g}$
- let us further assume that the objective function f is well behaved, say, *Lipschitz continuous* with constant L
- that is to say, for any $\boldsymbol{x}$ and $\boldsymbol{y}$, we have:

$$|f(\boldsymbol{x}) - f(\boldsymbol{y})| \leq L||\boldsymbol{x} - \boldsymbol{y}||.$$

4.2 Recurrent neural networks

- in this case, we can safely assume that, if we update the parameter vector by $\boldsymbol{g}$, then:

$$|f(\boldsymbol{x}) - f(\boldsymbol{x} - \eta \boldsymbol{g})| \leq L\eta||\boldsymbol{g}||,$$

  which means that we will not observe a change by more than $L\eta||\boldsymbol{g}||$
- this is both a curse and a blessing
- on the curse side, it limits the speed of making progress; whereas on the blessing side, it limits the extent to which things can go wrong if we move in the wrong direction

4.2 Recurrent neural networks

- sometimes, the gradients can be quite large, and the optimization algorithm may fail to converge
- we could address this by reducing the learning rate η
- but what if we only *rarely* get large gradients?
- in this case, such an approach may appear entirely unnecessary
- one popular alternative is to *clip* the *gradient* $\boldsymbol{g}$ by projecting it back to a ball of a given radius, say θ, via:

$$\boldsymbol{g} \leftarrow \min\left(1, \frac{\theta}{||\boldsymbol{g}||}\right)\boldsymbol{g}.$$

4.2 Recurrent neural networks

- by doing so, we know that the gradient norm never exceeds θ, and that the updated gradient is entirely aligned with the original direction of $\boldsymbol{g}$
- it also has the desirable side-effect of limiting the influence any given mini-batch (and within it, any given sample) can exert on the parameter vector
- this gives a certain degree of *robustness* to the model
- gradient clipping provides a quick fix to the gradient exploding
- while it does not entirely solve the problem, it is one of the many techniques to alleviate it

4.2 Recurrent neural networks

- recall that, for language modeling, in Section 4.1, we aim to predict the next token based on the current and past tokens, thus we shift the original sequence by one token as the labels
- Bengio et al. first proposed to use a neural network for language modeling, in 2003
- in the following, we illustrate how RNNs can be used to build a language model
- let the mini-batch size be 1, and the sequence of the text be “machine”
- to simplify things, we tokenize text into characters, rather than words, and consider a *character-level language model*
- Figure 4 demonstrates how to predict the next character based on the current and previous characters, via an RNN for character-level language modeling

4.2 Recurrent neural networks

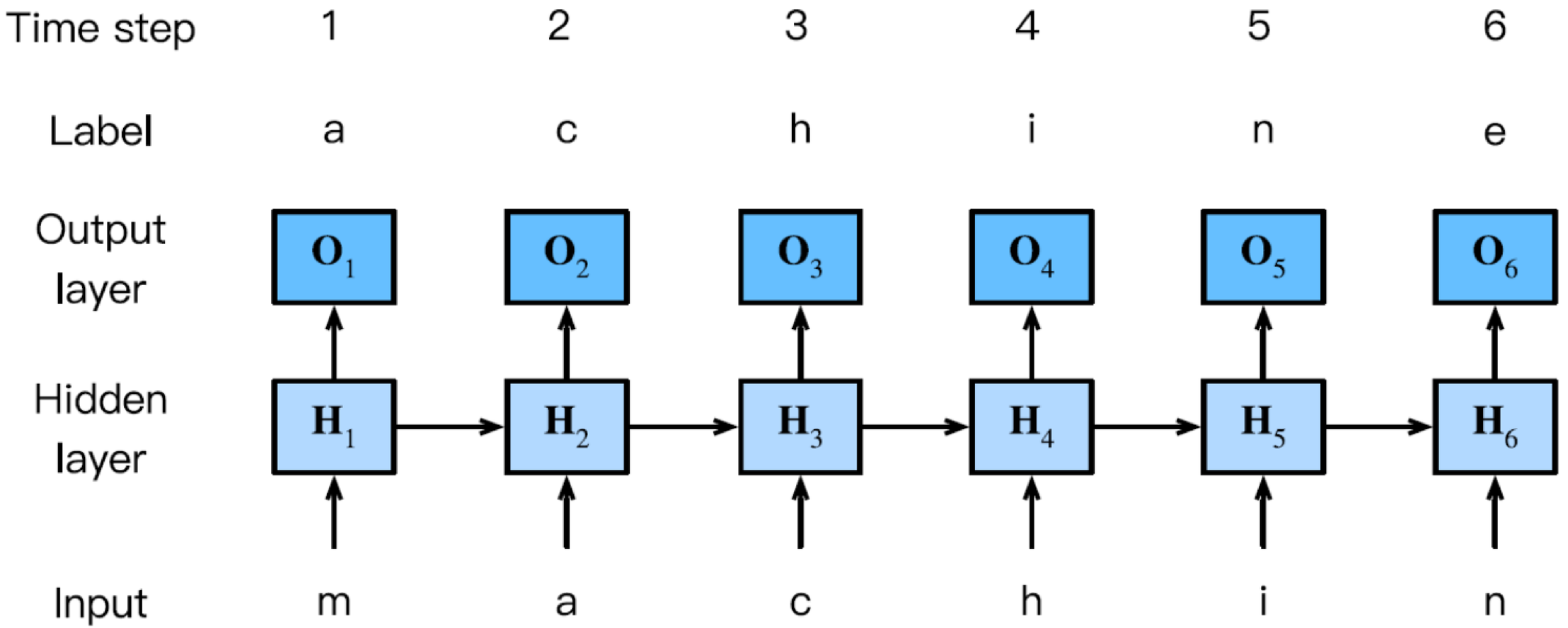


Figure 4: A character-level language model based on the RNN. The input and label sequences are “machin” and “achine”, respectively.

4.2 Recurrent neural networks

- during the training process, we run a softmax operation on the output from the output layer for each time step, and then use the cross-entropy loss to compute the error between the model output and the label
- due to the recurrent computation of the hidden state in the hidden layer, the output of time step 3 in Figure 4, $\boldsymbol{O}_3$, is determined by the text sequence “m”, “a”, and “c”
- since the next character of the sequence in the training data is “h”, the loss of time step 3 will depend on the probability distribution of the next character generated based on the feature sequence “m”, “a”, “c”, and the label “h” of this time step

4.2 Recurrent neural networks

- in practice, each token is represented by a d-dimensional vector, and we use a batch size $n > 1$
- therefore, the input $\boldsymbol{X}_t$ at time step t will be a $n \times d$ matrix, which is identical to what we discussed before
- lastly, let us discuss about how to measure the language model quality, which will be used to evaluate RNN-based models
- one way is to check how *surprising* the text is
- a good language model is able to predict with high accuracy tokens that we will see next
- consider the following continuations of the phrase “It is raining”, as proposed by different language models:
  1. “It is raining outside”
  2. “It is raining banana tree”
  3. “It is raining piouw;kcj pwepoiut”

4.2 Recurrent neural networks

- in terms of quality, example 1 is clearly the best; the words are logically coherent
- while it might not quite accurately reflect which word follows semantically (“in San Francisco” and “in winter” would have been perfectly reasonable extensions), the model is able to capture which kind of word follows
- example 2 is considerably worse, by producing a nonsensical extension
- nonetheless, at least the model has learned how to spell words, and some degree of correlation between words
- lastly, example 3 indicates a poorly trained model that does not fit the data properly

4.2 Recurrent neural networks

- we might measure the quality of the model by computing the likelihood of the sequence
- unfortunately, this is a number that is hard to understand and difficult to compare
- after all, shorter sequences are much more likely to occur than the longer ones, hence evaluating the model on a long novel will inevitably produce a much smaller likelihood than on a short novella
- what is missing is the equivalent of an average

4.2 Recurrent neural networks

- information theory is useful here; we have defined entropy and cross-entropy when we introduced the softmax regression, in Chapter 1
- if we want to compress text, we can ask about predicting the next token, given the current set of tokens
- a better language model should allow us to predict the next token more accurately
- thus, it should allow us to spend fewer bits in compressing the sequence

4.2 Recurrent neural networks

- so, we can measure it by the cross-entropy loss averaged over all the n tokens of a sequence:

$$\frac{1}{n}\sum_{t=1}^{n} -\log P(x_t|x_{t-1},\ldots,x_1), \tag{4}$$

  where P is given by a language model and x_t is the actual token observed at time step t from the sequence
- this makes the performance on documents of different lengths comparable
- for historical reasons, scientists in natural language processing prefer to use a quantity called *perplexity*
- in a nutshell, it is the exponential of (4):

$$\exp\left(-\frac{1}{n}\sum_{t=1}^{n}\log P(x_t|x_{t-1},\ldots,x_1)\right).$$

4.2 Recurrent neural networks

- perplexity can be best understood as the harmonic mean of the number of real choices that we have when deciding which token to pick next
- let us look at a number of cases:
  - In the best case scenario, the model always perfectly estimates the probability of the label token as 1. In this case, the perplexity of the model is 1.
  - In the worst case scenario, the model always predicts the probability of the label token as 0. In this situation, the perplexity is positive infinity.
  - At the baseline, the model predicts a uniform distribution over all the available tokens of the vocabulary. In this case, the perplexity equals the number of unique tokens of the vocabulary. In fact, if we were to store the sequence without any compression, this would be the best we could do to encode it. Hence, this provides a nontrivial upper bound that any useful model must beat.